Final Project: Design Your Own ISA

Himadri Saha

EECE.5821 Computer Architecture

12/9/2026

M1.1 Design Philosophy

This project uses a tiny load/store RISC design: 8-bit data words, 16-bit fixed-width instructions, and 8 general-purpose registers. Only load and store instructions access memory, and everything else works on registers. The goal is the fewest instructions and register fields that can still run all four benchmarks, so every opcode and register must be justified by the benchmarks it serves.

M1.2 Planned Parameters

Parameter	Quantity	Notes
Word Size	8 Bits	
Instruction Width	16 Bits (fixed length)	
Number of Registers	8 General Purpose registers (R0-R7)	
Memory Model	Instruction Memory: 256 Words of 16 bits read-only during execution Data Memory: 256 words of 8 bits	Separate Instruction and Data memory (Harvard style)
Address Width	8 bits for both memories Program Counter: 8 bits	
Estimated Number of Instructions	~12-14	

M1.3 Planned Instruction List

Instruction Name	Description
LD	Load a value from data memory into a register
ST	Store a register value into data memory
ADD	Add two registers and put the sum in a register
SUB	Subtract one register from another and put the result in a register
AND	Bitwise AND of two registers, result in a register
OR	Bitwise OR of two registers, result in a register
XOR	Bitwise XOR of two registers, result in a register
SHL	Shift a register left by one bit (logical, fill with 0)
SHR	Shift a register right by one bit (logical, fill with 0)
BEQ	Branch to a nearby address if two registers are equal
BNE	Branch to a nearby address if two registers are not equal
JAL	Jump to a subroutine and save the return address in a register

JR	Jump to the address stored in a register (used to return)
HALT	Stop the simulator

M1.4 Benchmark Plan

B1: Array Sum

```
XOR R1, R1, R1      ; ptr = 0
XOR R3, R3, R3      ; sum = 0
LD  R4, [8]         ; R4 = 1
LD  R5, [9]         ; R5 = 8 (end pointer)
loop:
LD  R2, [R1]        ; R2 = mem[ptr]
ADD R3, R3, R2      ; sum += value
ADD R1, R1, R4      ; ptr += 1
BNE R1, R5, loop    ; repeat until ptr == 8
ST  R3, [10]        ; store result (61 for test data)
HALT
```

B2: Multiply Shift-and-add (a in R1, b in R2)

```
LD  R1, [a_addr]    ; a
LD  R2, [b_addr]    ; b
LD  R6, [8]         ; R6 = 1 (bit mask)
XOR R3, R3, R3      ; product = 0
XOR R5, R5, R5      ; R5 = 0
loop:
BEQ R2, R5, done    ; stop when b == 0
AND R4, R2, R6      ; R4 = lowest bit of b
BEQ R4, R5, skip    ; bit is 0, skip the add
ADD R3, R3, R1      ; product += a
```

```
skip:
SHL R1, R1          ; a = a * 2
SHR R2, R2          ; b = b / 2
BEQ R5, R5, loop    ; always jump back
done:
HALT                ; standalone; becomes JR R7 in B3
```

B3: a·b + c·d using B2 twice

```
LD  R1, [a]
LD  R2, [b]
JAL R7, B2          ; call site 1: R3 = a*b
ST  R3, [tmp]       ; save first product
LD  R1, [c]
LD  R2, [d]
JAL R7, B2          ; call site 2: R3 = c*d
LD  R4, [tmp]
ADD R3, R3, R4      ; R3 = a*b + c*d (87 for test data)
ST  R3, [result]
HALT
```

M1.5 Pipeline Plan

Stage	What it does	Pipeline register written
IF (Instruction Fetch)	Reads the 16-bit instruction at the program counter from instruction memory and adds 1 to the program counter.	IF/ID
ID (Instruction Decode)	Decodes the opcode, reads the two source registers from the 8-register file, and sign-extends the branch offset.	ID/EX
EX (Execute)	Runs the ALU (ADD, SUB, AND, OR, XOR, SHL, SHR),	EX/MEM

	compares registers for BEQ and BNE, and computes branch and jump targets.	
MEM (Memory)	LD reads data memory and ST writes it. Other instructions pass their ALU result through.	MEM/WB

M1.6 Simulator Plan

Python will be used as it lets the pipleline be built and tested one stage at a time. Each pipleline register (IF/ID, ID/EX, EX/MEM, and MEM/WB) will be a class holding the instruction and the values and control signals that later stages need. Each simulated cycle will run the stages in the order WB, MEM, EX, ID, and IF, then copy the next pipeline registers into the current ones all at once. The simulator will have a functional mode that runs one whole instruction at a time, and a pipeline mode that counts cycles and reports CPI. A -trace flag will print one row per cycle showing the instruction in each stage, with bubbles and NOP-filled delay slots marked. Each benchmark will be checked by confirming that functional and pipeline modes end with the same registers and memory, starting with B1 and adding B2 and B3 as each one runs.

The plan includes the assembler extra credit. The assembler will be a separate Python program that reads the .asm files, resolves labels for branch targets and data, reports errors with line numbers, and writes the 16-bit machine code file that the simulator loads. B2 will still be hand-assembled for M2 as required.